Introduction and Summary

When creating an array on the stack, it is always recommended that the array be small and that its size is known at *compile* time (a constant). Usually the size of an array, once created, cannot be changed.

For large arrays or arrays whose size is not known until *runtime*, use dynamic allocation (from the heap using the **new** operator) and remember to delete them when they are no longer needed.

Project 1 – Dynamic Allocation of an Array (10 points)

Combine Programming Challenges #1 and #2 from our textbook into a single project (one set of programs).

Programming Challenge #1: Write a program which dynamically allocates an array large enough to hold a user-defined number of quiz or test scores. Once all scores are entered, the array should be passed to a function that sorts them in ascending order using a sorting algorithm that you write yourself. Another function (again, one which you should write) should be called that calculates the average score. The program should then display the sorted list of scores and the average with appropriate headings.

Use pointer notation rather than array notation whenever possible.

When finished with the array, use the **delete** operator to de-allocate the array.

Input Validation: Do not accept negative test scores

Do not submit anything for this part of the project.

Programming Challenge #2: Using the first project as a starting point, modify the program to allow the user to enter name-score pairs using a class that you define. For each student taking the quiz or test, the user should enter a string representing name of the student, followed by an integer representing the student's score. The name and score for each student should be stored in an object of a class in the array. (To keep things simple, this class can be defined in the same file as your **main** function.) In the class, make your instance variables private, so you'll also need to supply getters and setters.

The array from above should then be an array of class objects, rather than an array of **int**s. Modify both the sorting and average-calculating functions so they take arrays of *objects*. Sort the array by score. *Again, use pointers rather than square brackets wherever possible.*

Include a class definition for a student, including a no-argument constructor, a constructor which takes the student's name and score, getters and setters for the student's name and score, and a destructor. You may specify the class definition in the same file as **main** or in a separate file. (Please use a class and not a structure).

When finished with the array, use the **delete** operator to de-allocate the array of student objects.

Create sample output for at least two runs each with 4 or more student/score combinations. Print the report in columnar form with headings over the "name" and "score" columns.

Make sure to demonstrate the check for negative scores in your sample output, then submit the C++ source code for the final (combined) project and two sample outputs.

Project 2 - Dynamic Allocation of Two-Dimensional Arrays (15 points)

In CS 150, we learned that a two-dimensional array can be created on "the stack" very easily and initialized using an initialization list. For example, to create an array of 3 rows (height = 3) and 4 columns (width = 4), you might use something like this:

```
int a[3][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12} ;
```

Many prefer to use nested initialization lists which segregate the contents of each row:

```
int a[3][4] = { {1, 2, 3, 4},
                {5, 6, 7, 8},
                {9, 10, 11, 12} } ;
```

In both cases, the array will be stored in 12 consecutive **ints** in memory, first the contents of row 0, then row 1, and finally row 2. This is called "row-major" order.

Creating a 3x4 array on the stack

```
int a[3][4] = { {1, 2, 3, 4},
                {5, 6, 7, 8},
                {9, 10, 11, 12} } ;
```

	column 0	column 1	column 2	column 3
row 0	***a***[0][0] = 1	***a***[0][1] = 2	***a***[0][2] = 3	***a***[0][3] = 4
row 1	***a***[1][0] = 5	***a***[1][1] = 6	***a***[1][2] = 7	***a***[1][3] = 8
row 2	***a***[2][0] = 9	***a***[2][1] = 10	***a***[2][2] = 11	***a***[2][3] = 12

width = 4, height = 3

Two-dimensional array can be stored *dynamically* in one of two ways. Unfortunately, neither can use an initialization list.

1. The first way to store a *two*-dimensional array in the free-store is as a long *one*-dimensional array of consecutive rows in memory, one after the other, similar to the way it would be stored on the stack. To access elements in the array, only a single pointer and pointer arithmetic is used. An index into the one-dimensional array is computed as the row number (multiplied by the width) plus the column number of the two-dimensional array, Conversely, given an index into the 1-dimensional array, the corresponding row of the 2-dimensional array can be computed as **index / width,** and its column can be computed as **index % width**.

2. An alternative storage technique is to create a 2-dimensional dynamic array as an "array-of-arrays", where each row is allocated separately using the **new** operator. In this case, the array itself is also dynamically-allocated, and is an array of pointers to the one-dimensioned rows.

Simulating a 3x4 array with a 1-dimensional array

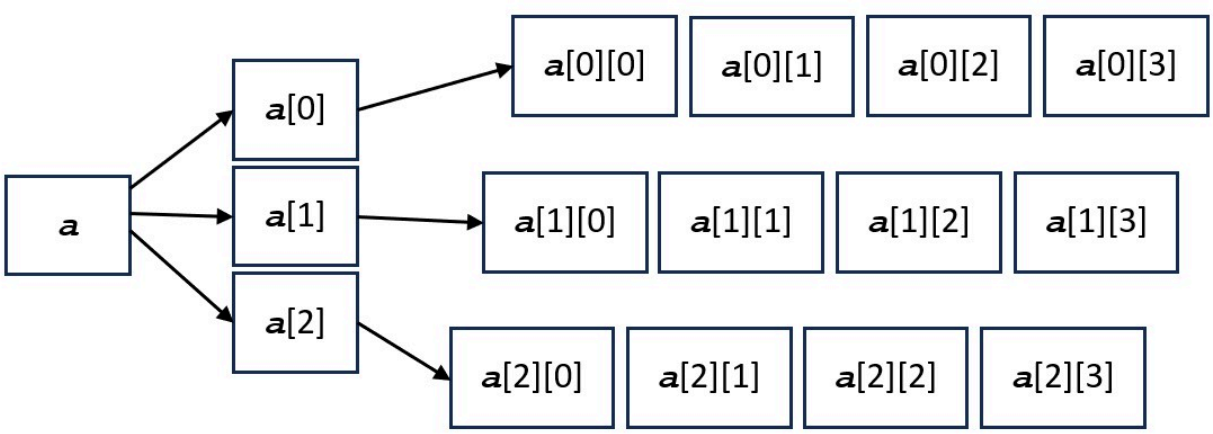
```
int *a = new int[height * width] ;
for (int row = 0 ; row < height ; row++) {
   for (int col = 0 ; col < width ; col++) {
      int index = (row * width) + col ;
      a[index] = index + 1 ;
   }
}
```

	column 0	column 1	column 2	column 3
row 0	a[0] = 1	a[1] = 2	a[2] = 3	a[3]= 4
row 1	a[4] = 5	a[5] = 6	a[6] = 7	a[7] = 8
row 2	a[8] = 9	a[9] = 10	a[10] = 11	a[11] = 12

row = index / width col = index % width

Creating a 3x4 “array of arrays” dynamically

```
int **a = new int * [height] ;
for (int row = 0 ; row < height ; row++)
   a[row] = new int[width] ;
```



width = 4, height = 3

Using this technique:

- each row is a separate array in memory, where column elements in each row are stored together.
- elements in the array can be accessed using the normal two-square-bracket notation: **a[row][column]**.
- when deleting the array, the individual rows should be de-allocated first, followed by the outer array:

```
for (int row = 0 ; row < height ; row++)
    delete[] a[row] ;     // delete one row
delete[] a ;              // delete top array
```

In this project, use both techniques to create dynamic two-dimensional arrays:

1. Create a *one*-dimensional dynamic array of 20 **ints** and fill with random numbers from 1 to 99.
2. Print the contents of the array, a grand total of the numbers in the array, and the average of those numbers to 2 decimal places.
3. Using the first technique described above for creating two-dimensional dynamic arrays, treat the one-dimensional array above as a two-dimensional array with 4 rows each with 5 numbers (height = 4, width = 5). Use the index into the first array (0 to 19) to calculate the associated row (0 to 3) and column (0 to 4) in a two-dimensional array. On separate lines, print the contents of each row followed by total and average of the numbers in that row.
4. Calculate a grand total of the numbers in the array using this technique of accessing elements in a 2-dimensional array (it should be the same as the grand total of all 20 numbers from step 2). Print this total aligned under the totals for each row.
5. Calculate the average of the four row averages. It should be the same as the average of all 20 numbers from step 2. Print this total aligned under the averages for each row.
6. Create a two-dimensional "array-of-arrays" using the second technique described above. The array should have 4 rows each with 5 columns.
7. Copy the numbers from the array used in steps 1-4 into the new array, where each element in the new two-dimensional array corresponds to the same element in the first two-dimensional array. You may use any technique you wish to copy the elements from one array to the other.
8. Similar to what you did in steps 3, 4, and 5, on separate lines print the contents of each row in the new array followed by the total and average of those numbers. They should match the numbers printed in steps 4 and 5 above for each row.
9. Deallocate the arrays created in steps 1 and 6.

Your output might look something like the following, though the numbers should be different:

```
            ** Original array of 20 numbers ***
 79 30 87 63 21 32 71 5 18 3 46 44 64 98 62 48 69 96 33 41
                Total: 1010  Average: 50.50

         *** Using that array as a 4 x 5 array ***
Row 0:  79  30  87  63  21   Total:  280  Average: 56.00
Row 1:  32  71   5  18   3   Total:  129  Average: 25.80
Row 2:  46  44  64  98  62   Total:  314  Average: 62.80
Row 3:  48  69  96  33  41   Total:  287  Average: 57.40
                        Grand Total: 1010  Average: 50.50

              *** Using an array-of-arrays ***
Row 0:  79  30  87  63  21   Total:  280  Average: 56.00
Row 1:  32  71   5  18   3   Total:  129  Average: 25.80
Row 2:  46  44  64  98  62   Total:  314  Average: 62.80
Row 3:  48  69  96  33  41   Total:  287  Average: 57.40
                        Grand Total: 1010  Average: 50.50
```

You do *not* need to use pointer notation exclusively in this project.

Make sure that the numeric columns in the report are properly aligned to the right using **printf**.

When finished, submit a copy of your C++ program and a snip of the output.

Project 3 – Pie à la "Mode" (5 points extra credit)

Programming Challenge #5: In statistics, the *mode* of a set of values is the value that occurs most often. Write a program which determines how many pieces of pie most people eat in a year.

Set up a dynamically-allocated integer array (using the **new** operator) that can hold responses from 15 people. For each person, enter the number of pieces of pie they say they eat in a year. Make sure this is a non-negative number and that several responses are repeated numbers (occur more than once). Then write a function that finds the mode of those 15 values, which will be the number of pie slices that the most people say they eat in a year. The function that finds and

returns the mode should accept two arguments, an array of integers and a value indicating how many elements are in the array.

To simplify this project, sort the array after all responses have been entered. Write the code to do the sorting yourself, do not use the **sort** method provided by C++.

Next, scan the sorted array, counting the number of times each response occurs consecutively in the array. Print the 15 responses in sorted order on the console to make verification easier.

In this project, please use pointer notation rather than square brackets wherever possible.

Provide your C++ source code and sample output of at least two different sequences of answers.

In each project, please remember to use the new form for opening comments in all homework assignments:

- *line 1*: name and purpose of program
- *line 2*: your name
- *line 3*: module number
- *line 4*: project number
- *lines 5-6*: problem statement
- *lines 7+*: algorithm

Always add plenty of comments in your code
and include an opening comment block in each program file.